Android Terminal Management Guide

The **Android Terminal Management Guide** explains how to manage Ingenico Moby5500 terminals using the TerminalManager class in the Android SDK. It focuses on three core capabilities:

- **Retrieve Terminal Information**: Access details such as the terminal's model, serial number, and firmware version.
- **Check for Firmware Updates**: Determine if newer firmware versions are available for the device.
- **Perform Firmware Upgrades**: Update the terminal's firmware to maintain security and ensure it stays current.

All operations require a properly connected terminal and valid credentials (e.g., bearer token). Developers must initialize the TerminalManager with the appropriate context and connection callbacks before performing any actions. Each function includes detailed steps and callback methods to ensure smooth integration and real-time feedback during terminal operations.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - Android Merchant Authorization & Accounts Guide
- **Merchant Terminal Connection Guide:** This step is required to connect the Ingenico Moby5500 terminal to your Android app.
  - Android Terminal Connection Guide

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Retrieving Terminal Information

To retrieve terminal information, ensure the terminal is properly connected and ready for communication. Once the connection is established, you can use the same TerminalManager instance to perform additional terminal operations.

Step 1: Create a TerminalManager object

ON THIS PAGE

Prerequisites

Retrieving Terminal Information

Checking for Available Firmware Updates

Performing Firmware Updates

```
)
```

Step 2: Call the getTerminalInformation() Method

Once the terminal is connected, use the existing **TerminalManager** object to call the following method:

```
terminalManager?.getTerminalInformation(context) // Pass the
application Context here
```

Step 3: Implement the Success Callback

If the method call getTerminalInformation is successful, the following callback is triggered:

```
override fun onRetrieveTerminalInformationSucceeded(terminalDetails:
TerminalInformation) {
// This callback is invoked when the method succeeds.
// It returns terminal details including:
// Firmware Version     = terminalDetails.firmwareInformation
// Battery Percentage   = terminalDetails.batteryPercentage
// Device Name          = terminalDetails.terminalName
// Device ID (Serial #) = terminalDetails.serialNumber
}
```

The triggered callback returns a **TerminalInformation** object that includes:

firmwareInformation: Represents the firmware version of the Ingenico device.

batteryPercentage: Represents current battery level of Ingenico device.

terminalName: Represents the bluetooth name of the terminal device.

serialNumber: Represents a unique identifier assigned to the Ingenico terminal device.

Step 4: Implement the Failure Callback

If the method **getTerminalInformation** fails, the following callback is triggered:

```
override fun onRetrieveTerminalInformationFailed(error: String) {
// This callback is invoked when the method fails to retrieve terminal
details.
// The 'error' parameter contains a description of the failure.
}
```

Checking for Available Firmware Updates

This process verifies whether the connected terminal is running the latest firmware by comparing its current version with the one available on the server. It ensures the device stays up to date with the latest features, enhancements, and security patches. If a newer version is detected, the system flags the need for an upgrade. The isUpgradeNeeded flag indicates whether an update is required, while availableVersion specifies the latest firmware version available for the terminal.

Step 1: Create and initialize the TerminalManager object

```
)
```

Note: To verify the firmware version of the connected terminal, ensure it is properly connected and online. The check will only work if the terminal is ready for communication.

Step 2: Initiate Firmware Version Check

Once the terminal is connected, use that TerminalManager object to initiate the firmware version check by calling the getAvailableFirmwareForUpgrade method.

```
terminalManager?.getAvailableFirmwareForUpgrade(context, getFirmware,
bearerToken, currentFirmwareVersion)
```

getFirmware: Contains Data class GetFirmware which provides the terminal's device name.

bearerToken: should be bearer token received from the MerchanAuthorizationService.

currentFirmwareVersion: Current firmware version of the connected terminal.

Step 3: Handling Successful Firmware Checks

If the check is successful, the onGetAvailableFirmwareForUpgradeSuccess callback is triggered. This callback provides information on whether an upgrade is needed, as well as the latest firmware version available for the device.

The callback parameters include:

availableVersion: The latest firmware version available on the server.

isUpgradeNeeded: A boolean flag indicating whether an upgrade is required.

```
override fun onGetAvailableFirmwareForUpgradeSuccess(
availableVersion: String, isUpgradeNeeded: Boolean)
{
//This callback is triggered after checking the terminal's firmware
version and
//indicates whether an upgrade is needed, along with the latest
available version.
}
```

Step 4: Handle Firmware Check Failure

If the firmware check fails, the onGetAvailableFirmwareForUpgradeFailure callback is invoked. This callback returns an error message detailing the reason for the failure.

```
override fun onGetAvailableFirmwareForUpgradeFailure(error: String)
```

status: TerminalConnectionStatus will be returned during failure.

- **TerminalConnectionStatus.getAvilableFirmwareForUpgradeFailed**: This is due to unexpected error.

error: The error parameter provides a message describing the reason for the failure.

Performing Firmware Updates

To keep the terminal updated with the latest firmware, initiate the firmware update process. This ensures the connected terminal downloads and installs the most recent firmware version available from the server. Before starting the update, make sure the terminal is properly connected and that valid firmware version details are provided—these are required for the process to begin successfully.

Step 1: Call the Firmware Update Method

```
terminalManager?.getAvailableFirmwareForUpgrade(     context,
getFirmware,
bearerToken,
currentFirmwareVersion )
```

context: Application context used for system operations.

bearerToken: should be bearer token received from the MerchanAuthorizationService.

currentFirmwareVersion: Current firmware version installed on the device (e.g., "14.10")

Step 2: Handle Firmware Upgrade Progress

This callback is invoked during the update process and delivers real-time progress updates, including the terminal's connection status, completion percentage, and a descriptive status message.

```
override fun onFirmwareUpdateProgress(terminalStatus:
TerminalHandlerConnectionStatus, progress: String, statusDescription:
String

{

}
```

terminalStatus: Represents the current TerminalConnectionStatus during the firmware upgrade process.

progress: Represents the percentage of upgrade completed.

statusDescription: Represents the current progress.

Step 3: Handle Firmware Update Success

This callback is triggered when the firmware update completes successfully. It confirms that the terminal has been updated and is now running the latest firmware version.

```
override fun onFirmwareUpdateSuccess() {
// Firmware update was completed successfully

}
```

status at the time of failure and a description of the error to assist with troubleshooting.

```
override fun onFirmwareUpdateFailed(terminalStatus:
TerminalHandlerConnectionStatus, error: String)
{
}
```

terminalStatus: Terminal status will be returned during failure. Some examples are:

- **TerminalConnectionStatus.firmwareDownloadFailed**: This is due to server or network issue.
- **TerminalConnectionStatus.firmwareUpdateFailed**: This is due to terminal hardware or unexpected error.

error: The error parameter provides a message describing the reason for the failure.

Note: *Please refer to the Complete SDK Reference for additional details Android SDK Reference Guide*

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation